Stellata

Business banking for small teams: technical and design overview

Case study for Post Oak Group · November 2026 · Live demo: https://stellata-banking-dashboard.vercel.app/ · Repository: github.com/dolorusso/stellata-banking-dashboard

1. Summary

Stellata is a banking dashboard for a small business. It shows the cash position across three accounts (Operating, Payroll, Tax Reserve), lets the owner search and export transactions, send ACH and wire transfers that settle in real time, and freeze a virtual debit card. It was built in 24 hours with Next.js and TypeScript, deploys to Vercel with zero configuration, and uses no database or paid services.

This document explains **what was built and why**: the name and brand, the visual direction, what each screen and chart shows, the technologies chosen, the architecture decisions, how the work was organized in branches, and how AI tools were used and verified. It ends with a proposed next feature: a natural-language assistant for asking questions about your money.

2. The name and the brand

Stellata comes from *Quercus stellata*, the botanical name of the **post oak**, a direct nod to Post Oak Group. In Latin, *stellata* means "star-covered", after the star-shaped hairs on the tree's leaves. The name gives the product its own identity while connecting it to the company it was built for.

- **The oak as the symbol.** Oaks stand for strength, longevity and slow, steady growth, which is exactly what a business wants from the place that holds its money. The logo is an original oak silhouette (a broad, lobed crown over a flared trunk), drawn from simple shapes so it still reads at 16 pixels as a favicon. It also appears as a faint watermark on the virtual card.
- **Brand color:** `#193e2c`, a deep oak green. Green is the color of money and growth, and a dark, desaturated green reads as established and calm rather than playful. It anchors the sidebar and the primary actions, and everything else stays neutral.
- **Original, not borrowed.** The brief asked for the quality of products like Mercury or Slash without copying them. The name, logo, palette and layouts are all original.

3. Design direction

The goal was a dashboard that feels **trustworthy, quiet and precise**, because people open a banking app to answer a question quickly and to feel in control. Every visual choice serves that:

Choice	Why
Neutral, slightly green-tinted surfaces with one brand accent	Lets the numbers be the loudest thing on screen. Color is reserved for meaning (money in/out, status) and for the one primary action.

Choice	Why
Branded deep-green sidebar on desktop; bottom navigation on mobile	Gives the product a recognizable identity without decorating content areas. The bottom nav keeps the four sections one thumb-tap away on phones.
Geist typeface; tabular numerals in tables, proportional figures in large totals	Tabular digits make columns of amounts line up so they can be compared at a glance. Large standalone totals use proportional digits, which look tighter at display sizes. The cents of the total balance are de-emphasized so the dollar figure reads first.
Light and dark themes, one-click toggle, no flash on load	Dark mode is expected in modern tools. Each theme was tuned separately (not an automatic inversion), and an inline script applies the saved theme before the first paint.
Restraint: thin chart marks, hairline gridlines, no gradients or heavy shadows	Dense financial screens get noisy fast. Restraint keeps attention on the data and makes the interface feel calm.
Accessibility as a design constraint	Every text and color pair was checked for WCAG AA contrast in both themes. Status always pairs an icon with a label (never color alone), everything works from the keyboard, and screen readers are told about async changes.

4. What the dashboard shows

4.1 Overview and KPIs

The overview answers "where do we stand?". It leads with the total balance, then the three accounts (with each one's share of the total), then four KPIs for the selected period. A single date-range selector (30 days, 90 days, 6 months) sits above everything it controls, so every number on the page always refers to the same period.

KPI	Definition	Why it's defined this way
Money in / Money out	Sum of inflows / outflows in the period	**Excludes transfers between your own accounts** (moving cash from Operating to Payroll is neither revenue nor spend; counting it would inflate both sides) and failed transactions. Pending transactions count, because that money is already committed.
Net cash flow	Money in minus money out	The simplest signal of whether the business is growing or shrinking its cash.
Runway	Total balance ÷ average monthly net burn	The question every small-business owner asks: how many months can we operate at this rate? When net flow is positive it shows "Cash-flow positive" instead of a meaningless number.
vs prior period	Change against the same-length period just before	A number without context is hard to judge. Arrows plus color show direction, and "up" is green only when up is good (more money out is shown as bad).

4.2 The charts

Each chart form was chosen for the question it answers. All three have tooltips, a legend when there is more than one series, and a **table view** (one click) with every value, so nothing is only readable by hovering.

Chart	What it answers	Why this form
Cash flow	In each period, how much came in and how much went out, and what was the net?	**Diverging bars**: money in grows up and money out grows down from a shared zero line, so the two are compared directly in the same column. Net is a dot per period rather than a line, because periods are discrete and a line would imply values in between. Daily bars for 30 days, weekly for 90 days, ~monthly for 6 months.
Spending by category	Where does our money go?	**Ranked horizontal bars in a single color**: bar length carries the magnitude and long category names stay readable. A donut was rejected because close values are hard to compare by angle. A sentence above the bars states the headline (e.g. payroll is 66% of spend).
Balance trend	How has our total cash moved over time?	**Area line of the end-of-day total**, zoomed to the data range with round axis ticks so changes are visible. Drawn with straight segments, because balances change in discrete steps and a smoothed curve would invent intermediate values.

> The categorical chart colors were **validated with a script** for color-vision deficiency in both themes. The first draft failed: the money-in green and money-out orange were nearly identical for people with protanopia. They were re-stepped to differ in lightness as well as hue. Animations are turned off on purpose: the charts refetch while transfers settle, and replaying an animation every few seconds would be distracting.

4.3 Transactions

About 220 realistic seeded transactions (Stripe payouts, client invoices, AWS, Gusto payroll, rent, IRS estimated taxes, card spend) over six months. Search, filters (date presets or custom range, category, status, account, amount range), sorting and pagination all run on the server, so they scale beyond what fits in the browser. Active filters appear as removable chips. A detail drawer shows method, reference (copyable), memo and expected settlement. **CSV export** downloads exactly the current filtered and sorted view, with amounts written as exact decimals and protection against spreadsheet formula injection. On mobile the table becomes a tappable list.

4.4 Transfers

A three-step flow: **details** → **review** → **result**. You can pay a saved recipient by ACH or wire, or move money instantly between your own accounts. Validation runs before the review step using the same rules as the server (enough funds, ACH daily limit, methods the recipient accepts), and errors point to the exact field. Review requires explicit confirmation and shows the balance after the transfer. After sending, the funds are held immediately and the transfer moves **pending** → **completed** live (about 8 seconds for a wire, 20 for ACH in the demo; same day and 1 to 2 business days in production). Balances, charts and the transaction list update everywhere on their own.

4.5 Cards

A virtual debit card with **freeze / unfreeze**. The switch updates instantly (optimistic update) and rolls back with an error message if the server call fails. The card dims and shows a "Frozen" badge, and the state survives a reload. A spend meter shows usage against the monthly limit (color plus text, never color alone), next to the card's recent activity.

5. Technologies

Technology	Role	Why it was chosen
Next.js 16 (App Router)	Framework: pages + API route handlers	One project for UI and API, first-class on Vercel with zero config. Uses Cache Components: pages are static shells and data loads inside Suspense boundaries.
TypeScript (strict)	Language	Strict mode plus `noUncheckedIndexedAccess` catches whole classes of bugs at compile time, which matters when the data is money.
Tailwind CSS v4	Styling	Design tokens as CSS variables (one source for both themes), fast iteration, no stylesheet sprawl.
shadcn/ui on Radix	UI primitives	Accessible building blocks (dialogs, selects, popovers, switches) with focus management handled correctly, owned as source code so they can be adjusted.
TanStack Query	Server state	Caching, loading and error states, and **polling only while a transfer is settling**, without hand-written effects.
TanStack Table v9	Transactions table	Headless: full control of markup and accessibility, with server-side sorting and pagination.
Recharts	Charts	Composable SVG charts with custom tooltips, styled through the same tokens.
Zod	Validation and types	**Single source of truth**: every type is inferred from a schema shared by client and server, and every API input is validated.
Faker (fixed seed)	Mock data	Deterministic, realistic data: same transactions every time, so the demo and the tests are stable.
Vitest	Unit tests	Fast tests for the parts where bugs are expensive: money math, transfer rules, balances, analytics.
Chrome DevTools Protocol script	Browser end-to-end checks	Drives a real Chrome or Edge with no extra dependencies: clicks, typing, keyboard, screenshots.
Vercel	Hosting	Push to main and it deploys. No environment variables needed.

6. Architecture and key decisions

6.1 A stateless backend that survives serverless

I chose API routes backed by an in-memory store, persisted in the browser, to demonstrate full-stack skills without needing a real database. On Vercel, consecutive requests can hit different serverless instances that don’t share memory, so server memory cannot be the source of truth. The design instead:The browser keeps a small **event log** in `localStorage`: the transfers the user made and each card's latest status. That is all the client stores.

- Every request sends that log **in the request body** (headers have size limits). The server rebuilds the full ledger from the deterministic seed plus the log, which is cheap at about 200 rows.
- Each event is **re-validated as it is replayed**. A tampered, duplicated or stale event is dropped, so the client can store its history but cannot forge a balance.
- Settlement is **derived from time** (created-at plus a delay per method), so there are no server timers to lose when an instance shuts down.

6.2 Other decisions

Decision	Reasoning
Money is always integer cents	Floating-point math is wrong for money (0.1 + 0.2 ≠ 0.3). Input is parsed from text straight to cents, cents become decimals only when displayed, and the formatter throws on a fractional cent so a float can't slip through silently.
Client runs the server's rules too	The transfer form validates with the same schema and business rules as the API for instant feedback, while the server stays authoritative. A rejection comes back as a 422 with a code and field, which the form maps to the right input.
Seed anchored to "today"	A fixed calendar date would leave "last 30 days" empty within weeks. The fixed random seed keeps the data identical, only the dates slide. Tests pin the date.
Accepted trade-offs for a demo	The event log is capped at 100 transfers (with a reset button) instead of snapshotting, the client can clear its own log (acceptable without authentication), and ACH settles in seconds instead of days.

7. How I worked: vertical slices and branches

The project was built in **six vertical slices**, each delivering something working end to end, each on its own branch and merged into `main` with `--no-ff`, so the history shows one merge per slice:

Branch	What it delivered
`main` (first commit)	`CLAUDE.md`: the conventions, written and approved before any code
`feat/a-scaffold`	Project setup, design tokens, theme, app shell and navigation
`feat/b-data-api`	Data model, money utilities, deterministic seed, ledger, API routes and their tests

Branch	What it delivered
`feat/c-overview`	Overview: balances, KPIs, the three charts and the range selector
`feat/d-transactions`	Transactions table, filters, detail drawer, CSV export
`feat/e-transfers-cards`	Transfer flow and card controls (built by two agents in parallel), versioned subagents
`feat/f-polish`	Browser end-to-end suite, 404 and error pages, oak logo, README, final review
`fix/theme-toggle`	One-click light/dark toggle replacing the theme menu

Every slice ended the same way: typecheck, lint, unit tests and production build all passing, the slice checked in a real browser, a short summary, then a commit with a conventional message (`feat:`, `fix:`, `chore:`). **I ran every git operation myself.** The AI proposed the commands and messages, and I reviewed each slice before committing and merging. Working this way keeps every step reviewable and makes it easy to see when and why each part was added.

8. AI tools and how I used them

The project was built with **Claude Code** (Anthropic's coding agent, running Claude Opus 5.5) as a pair programmer. The aim was not just speed but a **repeatable, reviewable workflow**: conventions written down, work split into clear roles, and every result verified by running the app. The full record is in `docs/ai-workflow.md` in the repository.

8.1 What I decided and what I delegated

- **I decided:** the plan and its approval gate, the architecture (for example, sending the event log in the request body instead of a header), the name and brand color, the branch-per-slice workflow, scope changes (no screenshots in the repo, a two-mode theme toggle), and every commit and merge.
- **I delegated:** writing code against the agreed conventions, running checks, driving the browser, and drafting documentation, always reviewing the result.

8.2 Conventions first: CLAUDE.md

Before any code, `CLAUDE.md` set the rules that would be expensive to get wrong later: money in integer cents, Zod as the source of truth, the API contract, folder structure, accessibility, loading/empty/error states, and design tokens only. It was updated whenever a decision changed, so it stays an accurate brief for any person or agent who picks up the repo.

8.3 Subagents, defined as files in the repository

Two subagent roles live in `.claude/agents/`, versioned with the code, each with an explicit tool list:

Agent	Tools	Role and result
`feature-slice-builder`	Read, Write, Edit, Search, Bash	Builds one feature in parallel with others under a **strict file-ownership contract**: it can only touch its own folders, never shared code, never git or the build, and it reports any shared change it needs. Two ran in parallel to build Transfers and Cards. The Cards builder reported a real gap: the server had no card filter, so older card activity was invisible. I added the filter and its test.
`convention-reviewer`	Read, Search, Bash (**no write tools**)	Audits each slice against `CLAUDE.md` before the commit. It cannot change code, only report. It caught, among others, that **screen readers would have re-announced the transfer countdown every second**, and it verified every factual claim in the README against the code.

8.4 Verification over trust

Every claim of "it works" was checked. Besides typecheck, lint and 79 unit tests, the AI drove a real headless browser, clicked through each flow, took screenshots and inspected them. The bugs that mattered were found this way, not by reading code:

Found by	Bug	Fix
Unit test	The seed created "today" transactions with times that hadn't happened yet	History ends yesterday
API smoke test	The CSV safety guard turned negative amounts into text in Excel	Plain numbers are exempt
Palette validator	Money in/out colors indistinguishable for red-green color blindness	Re-stepped by lightness, re-validated
Browser test	Closing the detail drawer sent keyboard focus to the top of the page	Focus returns to the row
Screenshot	Two countdowns for the same transfer disagreed (~8s vs ~13s)	Clock syncs immediately
Reviewer agent	Countdown re-announced every second to screen readers	Announce status changes only
Reading the test output	A test passed while the screen showed the wrong number (an operator-precedence bug in the test itself)	Fixed the test so it fails when it should

> A deliberate habit: the reviewer once flagged something it could not confirm statically (a page title). The browser showed it was fine. **Static review proposes; running the app decides.** The same applies to AI output in general: Next.js 16 and TanStack Table v9 differ from older patterns, so the bundled documentation was read before writing code instead of relying on memory.

9. Quality at a glance

- **79 unit tests** (Vitest): money parsing and formatting edge cases, transfer rules (one cent over the balance, daily limits, unsupported methods), ledger invariants (the seed never overdraws, internal transfers conserve the total, tampered and duplicate events are dropped), analytics consistency, CSV escaping.
- **42 browser checks** (`npm run e2e`): search, sort, filters, pagination, keyboard access, a full wire transfer (the total balance drops by exactly $1,500.00), card freeze surviving a reload, theme toggle, reset, no console errors, and no horizontal scrolling on a 390px phone.
- **Accessibility:** labelled inputs, keyboard-reachable everything, focus moved to the right place in multi-step flows, status never shown by color alone, AA contrast verified for every color pair.

10. Proposed next feature: "Ask Stellata"

A natural-language assistant inside the dashboard, so an owner can **ask questions about their money in plain words** instead of building filters. For example:

- "What was my last transaction?"
- "How much did we spend on software last month?"
- "Which vendor did we pay the most in Q3?"
- "Did Acme Robotics' invoice payment arrive?"
- "Has the wire to Brightline Design settled yet?"
- "List pending transactions over $500."

How I would build it

1. **A chat panel** (a drawer opened from the header) that streams the answer as it is written, so the reply starts appearing within a second.
2. **A server route** (`POST /api/assistant`) that calls the **Claude API** with tool use (Claude Opus 5.5 through Anthropic's TypeScript SDK). The model never sees raw data up front and never does the math itself: it decides which question to ask the app.
3. **Tools that wrap the existing API**: `search_transactions`, `get_overview`, `list_transfers`, `get_accounts`, `get_cards`. Their input schemas come from the **same Zod schemas** the dashboard already uses, so every tool call is validated exactly like a request from the UI. All amounts stay in integer cents and are formatted by the app.
4. **Answers with receipts**: each answer links to the underlying view (for example, the transactions table with the exact filters applied), so the user can verify the numbers.
5. **Read-only first.** A later phase could draft a transfer ("send $2,000 to Fernhill by ACH"), but it would only pre-fill the existing review step: the user still confirms, and the same server rules apply.

Guardrails

- **Scoped data access:** tools only return the signed-in business's data. The model can't reach anything the user couldn't see in the UI.
- **Prompt caching** of the system prompt and tool definitions keeps cost and latency low on repeated questions.

- **An evaluation set** of real questions with expected answers (like the ten above), run on every change, so accuracy is measured rather than assumed.
- **Audit logging and rate limits** for every assistant request, consistent with the rest of the roadmap.

11. What I would do next

1. **Real authentication and roles** (owner, admin, bookkeeper), approval rules for large transfers, and two-factor confirmation before moving money.
2. **A banking-as-a-service provider** (for example Unit, Treasury Prime, Column or Increase) instead of the mock ledger, with webhooks driving status changes instead of time-based settlement.
3. **A real ledger database**: Postgres with an append-only, double-entry journal. The event-replay design maps onto it directly.
4. **Audit logs, idempotency keys and rate limiting** on money movement, so a retried request can never send money twice.
5. **Observability and CI**: structured logs, error tracking, and the end-to-end checks running on every pull request.